

11.1-AI-ASSIGNMENT

2303A52249

A.VAMSHI KRISHNA

TASK-1

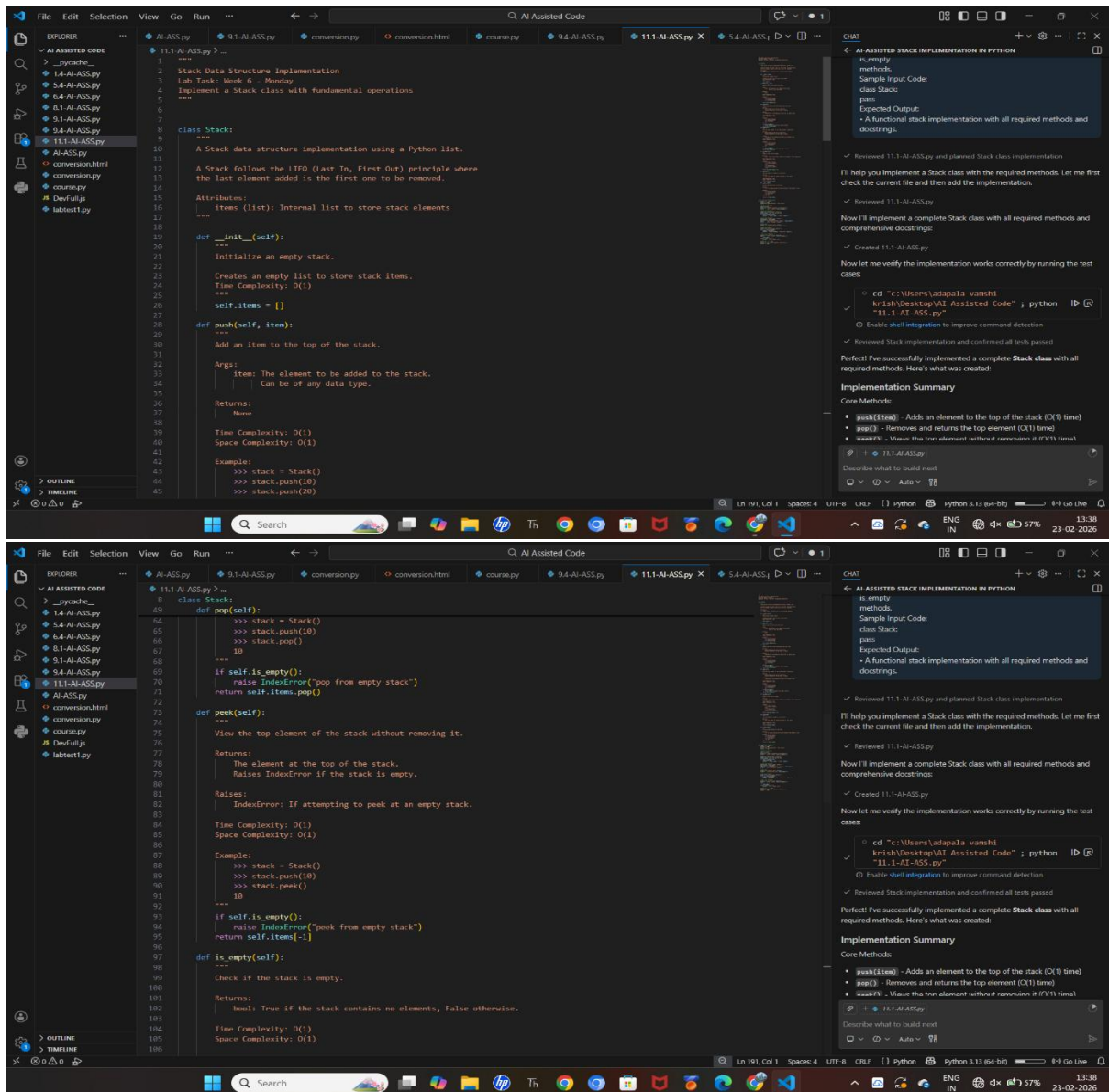
Scenario:

In this task, we used AI to design and implement a Stack data structure in Python. The objective was to understand how a stack follows the LIFO (Last In, First Out) principle and how AI can assist in writing structured, optimized, and well-documented code. Instead of manually implementing the class, AI was prompted to generate a complete and functional Stack with essential operations.

Prompt:

“Generate a beginner-friendly Python Stack class using a list. Include push, pop, peek, and is_empty methods. Add proper docstrings and handle edge cases like popping from an empty stack.”

Code:



```
class Stack:
    """
    A Stack data structure implementation using a Python list.

    A Stack follows the LIFO (Last In, First Out) principle where
    the last element added is the first one to be removed.

    Attributes:
        items (list): Internal list to store stack elements
    """
    def __init__(self):
        """
        Initialize an empty stack.

        Creates an empty list to store stack items.
        Time Complexity: O(1)
        """
        self.items = []

    def push(self, item):
        """
        Add an item to the top of the stack.

        Args:
            item: The element to be added to the stack.
                  Can be of any data type.

        Returns:
            None
        """
        self.items.append(item)

    def pop(self):
        """
        Remove the top element from the stack.

        Returns:
            The element removed from the stack.
        """
        if self.is_empty():
            raise IndexError("pop from empty stack")
        return self.items.pop()

    def peek(self):
        """
        View the top element of the stack without removing it.

        Returns:
            The element at the top of the stack.
            Raises IndexError if the stack is empty.

        Raises:
            IndexError: If attempting to peek at an empty stack.

        Time Complexity: O(1)
        Space Complexity: O(1)

        Example:
            >>> stack = Stack()
            >>> stack.push(10)
            >>> stack.push(20)
            >>> stack.peek()
            20
        """
        if self.is_empty():
            raise IndexError("peek from empty stack")
        return self.items[-1]

    def is_empty(self):
        """
        Check if the stack is empty.

        Returns:
            bool: True if the stack contains no elements, False otherwise.

        Time Complexity: O(1)
        Space Complexity: O(1)
        """
        return len(self.items) == 0
```

AI ASSISTED STACK IMPLEMENTATION IN PYTHON

is_empty methods.

Sample Input Code:

```
class Stack:
    pass
```

Expected Output:

- A functional stack implementation with all required methods and docstrings.

✓ Reviewed 11.1-AI-ASS.py and planned Stack class implementation

I'll help you implement a Stack class with the required methods. Let me first check the current file and then add the implementation.

✓ Reviewed 11.1-AI-ASS.py

Now I'll implement a complete Stack class with all required methods and comprehensive docstrings:

✓ Created 11.1-AI-ASS.py

Now let me verify the implementation works correctly by running the test cases:

```
cd "C:\Users\adapala.vamshi\krish\Desktop\AI Assisted Code" ; python "11.1-AI-ASS.py"
```

✓ Enable shell integration to improve command detection

✓ Reviewed Stack implementation and confirmed all tests passed

Perfect! I've successfully implemented a complete Stack class with all required methods. Here's what was created:

Implementation Summary

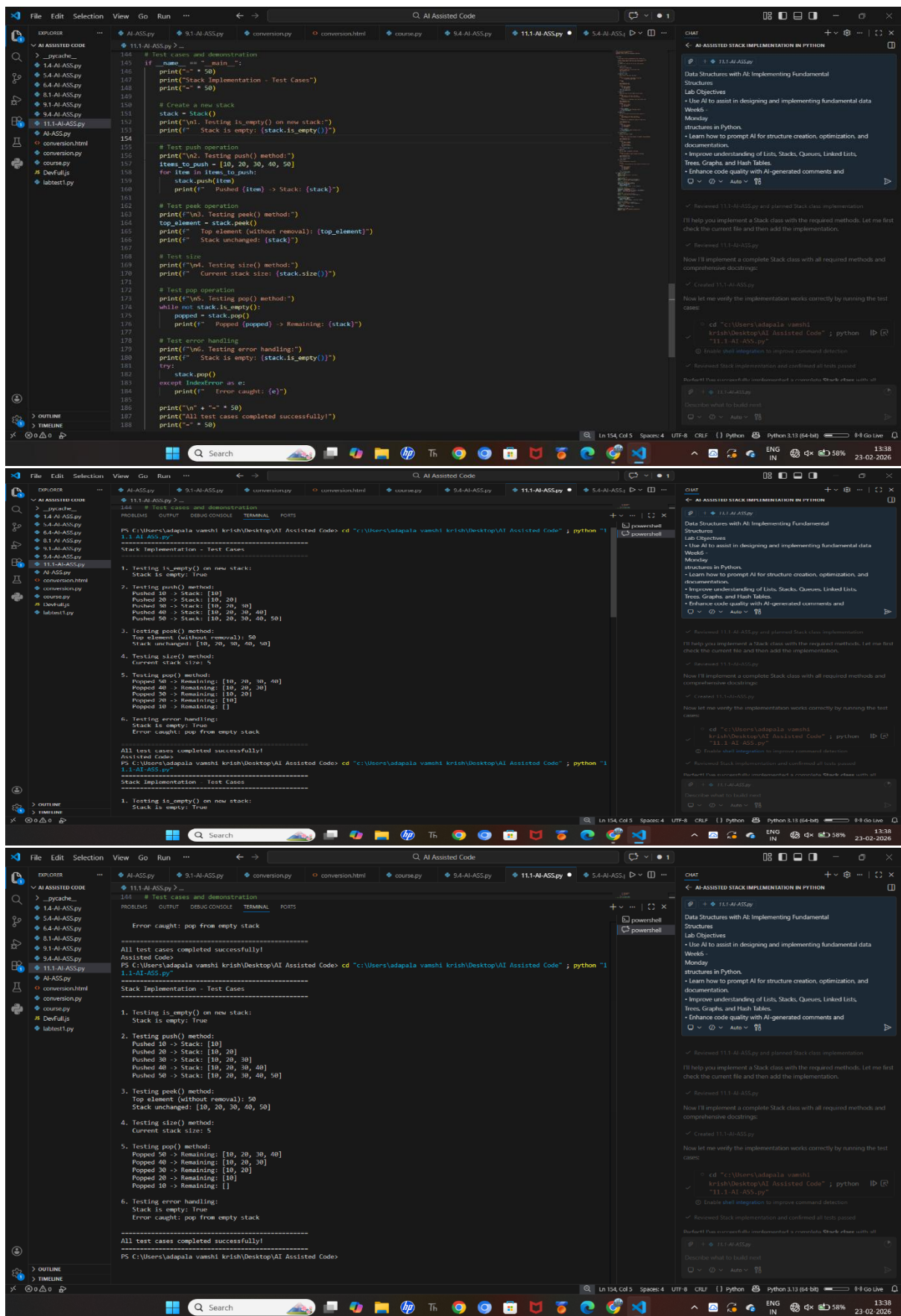
Core Methods:

- **push(item)** - Adds an element to the top of the stack (O(1) time)
- **pop()** - Removes and returns the top element (O(1) time)
- **peek()** - Views the top element without removing it (O(1) time)
- **is_empty()** - Checks if the stack is empty (O(1) time)

Describe what to build next

11.1-AI-ASS.py

Auto



Observation:

The AI successfully generated a functional Stack implementation with all required methods and proper documentation. It

handled edge cases using exceptions and used efficient list operations with $O(1)$ time complexity. This task demonstrated that AI can speed up development, improve code readability, and help in better understanding fundamental data structures.

TASK-2

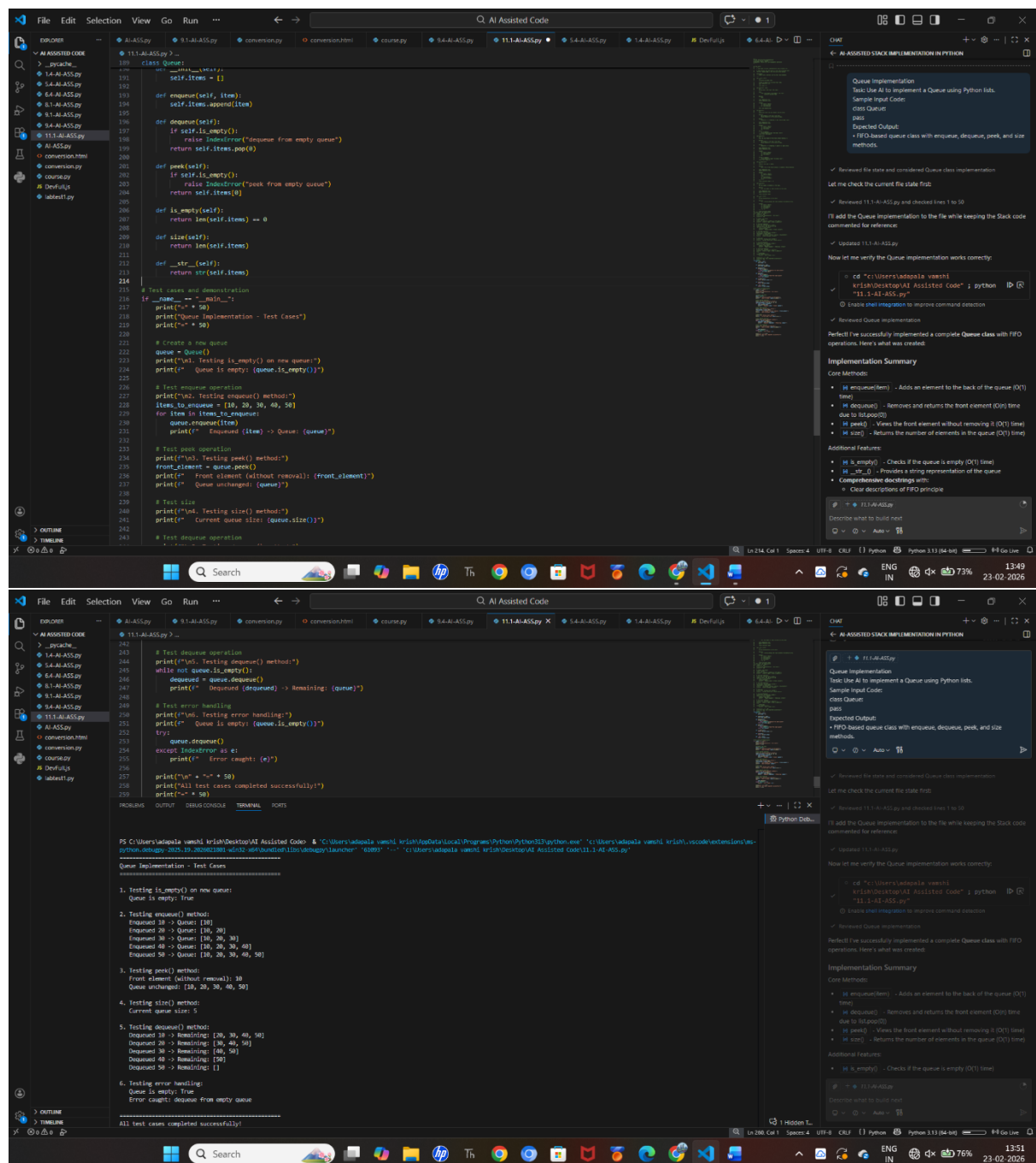
Scenario:

In this task, we used AI to implement a Queue data structure in Python using lists. The goal was to understand how a queue follows the FIFO (First In, First Out) principle and how AI can help in generating structured, clean, and efficient code. AI was prompted to create a functional Queue class with essential operations and proper handling of edge cases.

Prompt:

“Generate a beginner-friendly Python Queue class using a list. Include enqueue, dequeue, peek, and size methods. Follow FIFO principle and handle edge cases like dequeue from an empty queue. Add proper docstrings.”

Code:



```
11.1-AI-Assay.py
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if self.is_empty():
            raise IndexError("dequeue from empty queue")
        return self.items.pop(0)

    def peek(self):
        if self.is_empty():
            raise IndexError("peek from empty queue")
        return self.items[0]

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

    def __str__(self):
        return str(self.items)

# Test cases and demonstration
if __name__ == "__main__":
    print("\n Queue Implementation - Test Cases")
    print("\n" * 50)

    # Create a new queue
    queue = Queue()
    print("\n1. Testing is_empty() on new queue:")
    print(f"Queue is empty: {queue.is_empty()}")

    # Test enqueue operation
    print("\n2. Testing enqueue() method:")
    items_to_enqueue = [10, 20, 30, 40, 50]
    for item in items_to_enqueue:
        queue.enqueue(item)
    print(f"Enqueued {len(items_to_enqueue)} items to Queue: {queue}")

    # Test peek operation
    print("\n3. Testing peek() method:")
    front_element = queue.peek()
    print(f"Front element (without removal): {front_element}")
    print(f"Queue unchanged: {queue}")

    # Test size
    print("\n4. Testing size() method:")
    print(f"Current queue size: {queue.size()}")

    # Test dequeue operation
    print("\n5. Testing dequeue() method:")
    while not queue.is_empty():
        dequeued = queue.dequeue()
        print(f"Dequeued {dequeued} -> Remaining: {queue}")

    # Test error handling
    print("\n6. Testing error handling:")
    print(f"Queue is empty: {queue.is_empty()}")
    try:
        queue.dequeue()
    except IndexError as e:
        print(f"Error caught: {e}")

    print("\n" * 50)
    print("All test cases completed successfully!")
    print("\n" * 50)
```

Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if self.is_empty():
            raise IndexError("dequeue from empty queue")
        return self.items.pop(0)

    def peek(self):
        if self.is_empty():
            raise IndexError("peek from empty queue")
        return self.items[0]

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

    def __str__(self):
        return str(self.items)
```

Expected Output:

```
Queue Implementation
Task: Use AI to implement a Queue using Python lists.
Sample Input Code:
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if self.is_empty():
            raise IndexError("dequeue from empty queue")
        return self.items.pop(0)

    def peek(self):
        if self.is_empty():
            raise IndexError("peek from empty queue")
        return self.items[0]

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

    def __str__(self):
        return str(self.items)
```

Perfect! I've successfully implemented a complete Queue class with FIFO operations. Here's what was created:

Implementation Summary

Core Methods:

- `enqueue(item)` - Adds an element to the back of the queue ($O(1)$ time)
- `dequeue()` - Removes and returns the front element ($O(1)$ time due to list pop(0))
- `peek()` - Views the front element without removing it ($O(1)$ time)
- `size()` - Returns the number of elements in the queue ($O(1)$ time)

Additional Features:

- `is_empty()` - Checks if the queue is empty ($O(1)$ time)
- `__str__()` - Provides a string representation of the queue
- Comprehensive docstrings with:
- Clear descriptions of FIFO principle

PS C:\Users\adamsa\vscode\11.1-AI-Assay> python 11.1-AI-Assay.py

Queue Implementation - Test Cases

```
1. Testing is_empty() on new queue:
Queue is empty: True

2. Testing enqueue() method:
Enqueued 10 -> Queue: [10]
Enqueued 20 -> Queue: [10, 20]
Enqueued 30 -> Queue: [10, 20, 30]
Enqueued 40 -> Queue: [10, 20, 30, 40]
Enqueued 50 -> Queue: [10, 20, 30, 40, 50]

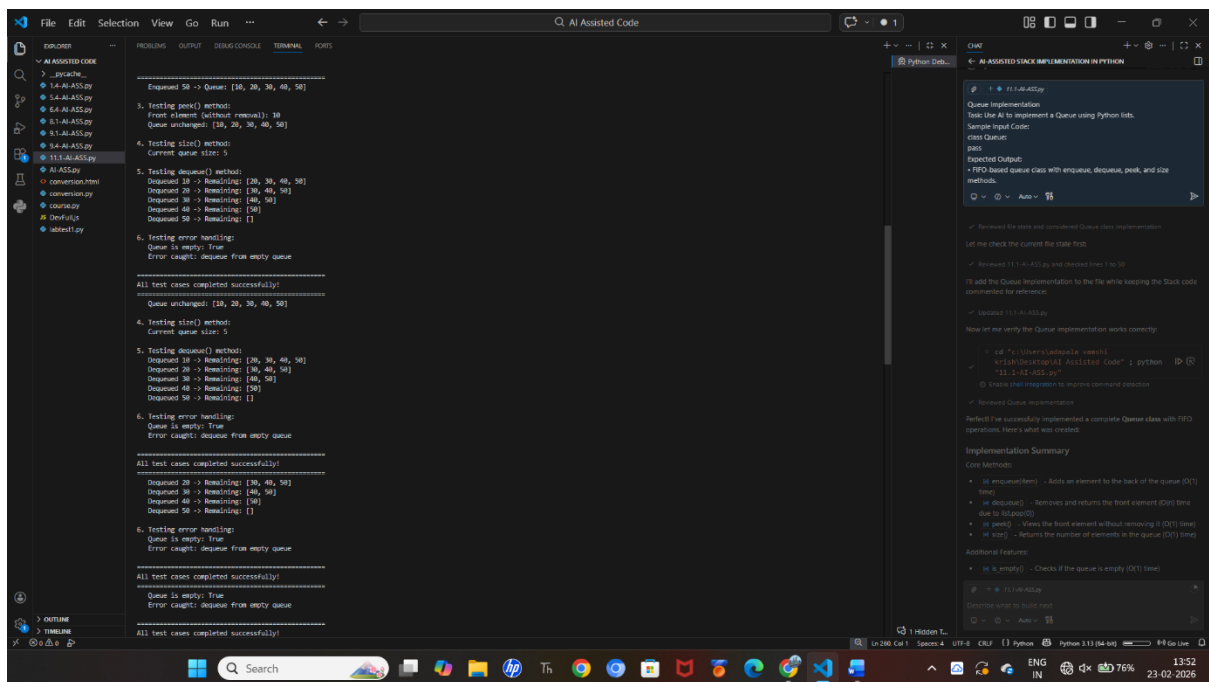
3. Testing peek() method:
Front element (without removal): 10
Queue unchanged: [10, 20, 30, 40, 50]

4. Testing size() method:
Current queue size: 5

5. Testing dequeue() method:
Dequeued 10 -> Remaining: [20, 30, 40, 50]
Dequeued 20 -> Remaining: [30, 40, 50]
Dequeued 30 -> Remaining: [40, 50]
Dequeued 40 -> Remaining: [50]
Dequeued 50 -> Remaining: []

6. Testing error handling:
Queue is empty: True
Error caught: dequeue from empty queue

All test cases completed successfully!
```



Observation:

The AI generated a functional FIFO-based Queue class with all required methods. It properly handled empty queue conditions using exceptions and included documentation for clarity. The enqueue operation works efficiently, while dequeue from a list may take $O(n)$ time due to element shifting. This task shows that AI can assist in writing clear, structured implementations of fundamental data structures and improve code quality.

TASK-3

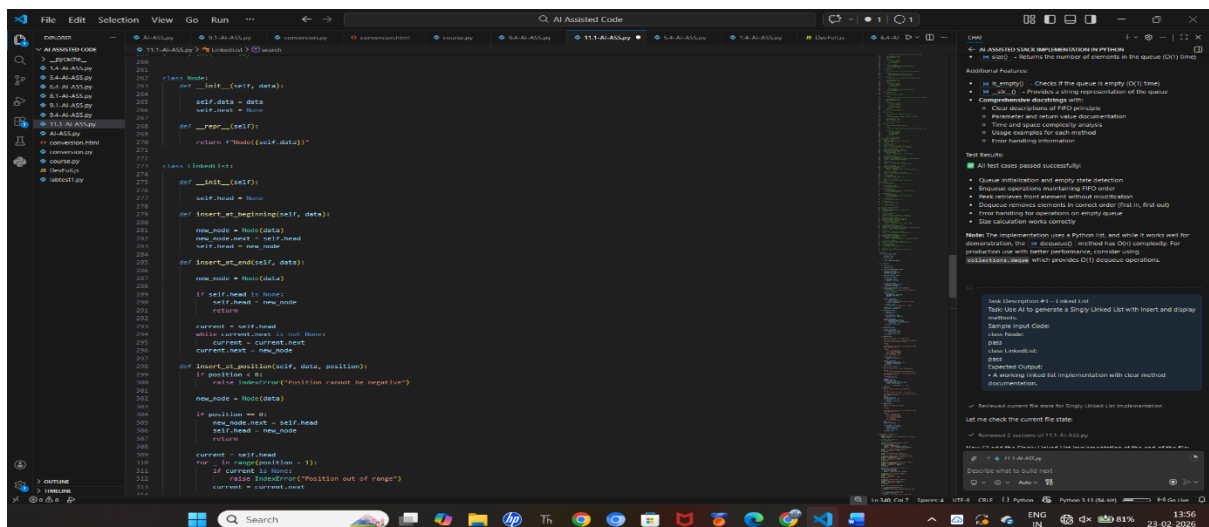
Scenario:

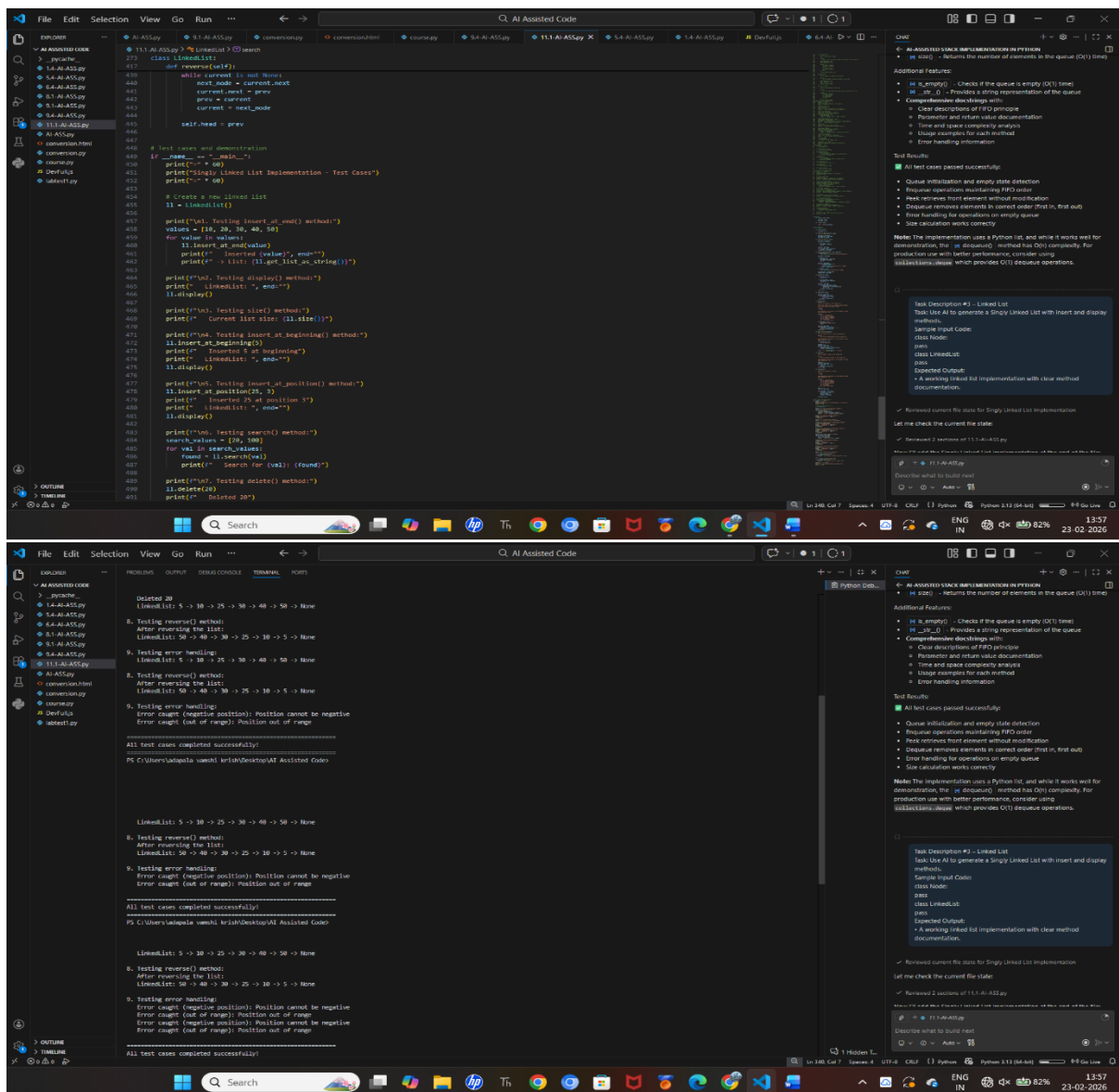
In this task, we used AI to generate a Singly Linked List implementation in Python. The objective was to understand how linked lists work using nodes and pointers instead of built-in lists. AI was used to create a Node class and a LinkedList class with insert and display methods, along with proper documentation for better understanding and readability.

Prompt:

“Generate a beginner-friendly Python implementation of a Singly Linked List. Create a Node class and a LinkedList class. Include insert and display methods. Add clear docstrings and keep the implementation simple and well-structured.”

Code:





Observation:

The AI successfully generated a working Singly Linked List with proper class structure and method documentation. The insert method correctly adds new nodes, and the display method traverses and prints the list elements. The implementation clearly demonstrates how nodes are connected using references, improving understanding of dynamic memory structures. This task shows how AI can help simplify complex data structure implementations and improve code clarity.

TASK-4

Scenario:

In this task, we used AI to implement a Binary Search Tree (BST) in Python. The objective was to understand how BST maintains sorted order using the property that left child values are smaller and right child values are larger than the root. AI was used to generate a BST class with recursive insert and in-order traversal methods to demonstrate how elements are stored and retrieved in sorted order.

Prompt:

“Generate a beginner-friendly Python implementation of a Binary Search Tree (BST). Include recursive insert and in-order traversal methods. Keep the structure simple and add proper docstrings for clarity.”

Code:

Observation:

The AI successfully generated a BST implementation with recursive insert and in-order traversal methods. The insert method correctly places nodes according to BST rules, and the in-order traversal prints elements in sorted order. The recursive approach improves understanding of tree traversal logic. This task demonstrates that AI can effectively assist in implementing hierarchical data structures with clear and well-documented code.

TASK-5

Scenario:

In this task, we used AI to implement a Hash Table in Python with basic operations like insert, search, and delete. The objective was to understand how hashing works for fast data access and how collisions are handled using chaining (storing multiple elements in the same bucket). AI was used to generate a structured and well-documented implementation to clearly demonstrate these concepts.

Prompt:

“Generate a beginner-friendly Python HashTable class. Include insert, search, and delete methods. Implement collision handling using chaining (lists inside buckets). Add clear comments and keep the implementation simple.”

Code:

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        # Each slot contains a list (chain) for handling collisions
        self.table = [[] for _ in range(size)]

    def _hash_function(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self._hash_function(key)
        chain = self.table[index]

        # Check if key already exists and update its value
        for i, (k, v) in enumerate(chain):
            if k == key:
                chain[i] = (key, value)
                return

        # Key doesn't exist, add new pair to the chain
        chain.append((key, value))

    def search(self, key):
        index = self._hash_function(key)
        chain = self.table[index]

        # Search through the chain for the key
        for k, v in chain:
            if k == key:
                return v
        return None

    def delete(self, key):
        index = self._hash_function(key)
        chain = self.table[index]

        # Search and remove the key from the chain
        for i, (k, v) in enumerate(chain):
            if k == key:
                chain.pop(i)
                return True
        return False

    def contains(self, key):
        return self.search(key) is not None

# Create a hash table
ht = HashTable(size=10)

print("\n. Testing insert() method:")
pairs = [
    ("name", "Alice"),
    ("age", 25),
    ("city", "New York"),
    ("email", "alice@example.com"),
    ("phone", "555-1234"),
    ("country", "USA"),
    ("job", "Engineer")
]
for key, value in pairs:
    ht.insert(key, value)
    print(f"Inserted {key}: {value}")

ht.display()

print("\n. Testing search() method:")
search_keys = ["name", "city", "country", "unknown"]
for key in search_keys:
    result = ht.search(key)
    print(f"Search for '{key}': {result}")

print("\n. Testing contains() method:")
print(f"Contains 'age': {ht.contains('age')}")
print(f"Contains 'height': {ht.contains('height')}")

print("\n. Testing update operation:")
ht.insert("age", 28)
print(f"Updated 'age' to 28")
print(f"Search for 'age': {ht.search('age')}")

print("\n. Testing delete() method:")
keys_to_delete = ["email", "phone", "unknown"]
for key in keys_to_delete:
    result = ht.delete(key)
    print(f"Deleted '{key}': {result}")

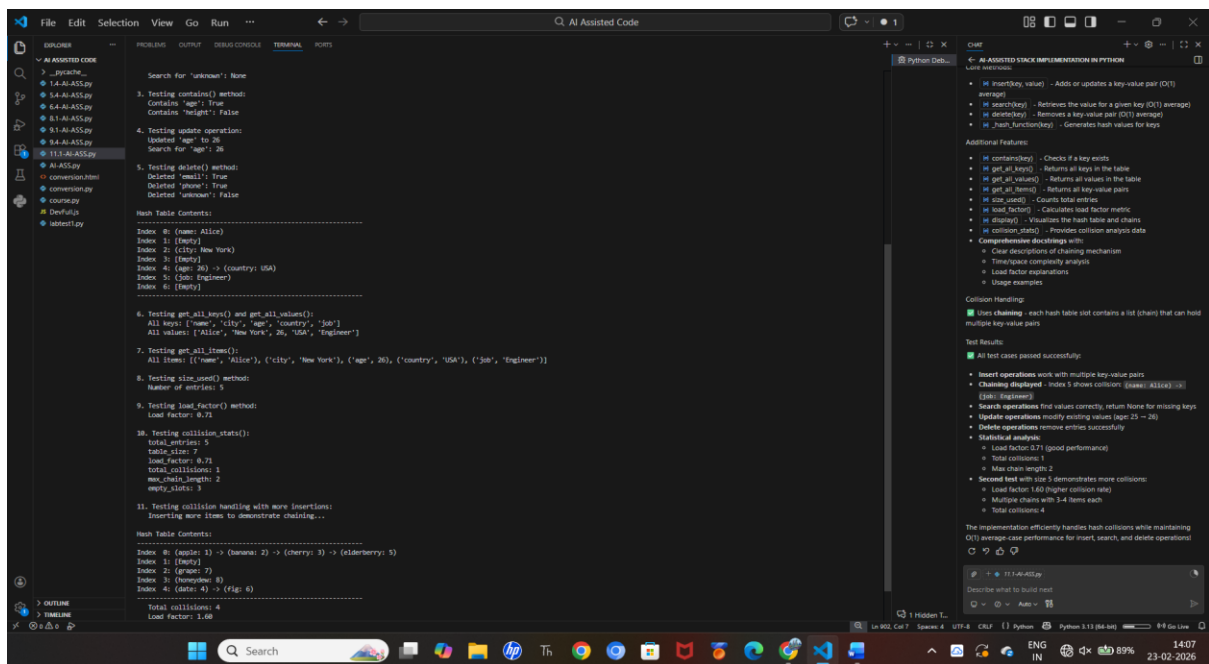
ht.display()

print("\n. Testing get_all_keys() and get_all_values():")
print(f"All keys: {ht.get_all_keys()}")
print(f"All values: {ht.get_all_values()}")

print("\n. Testing get_all_items():")
print(f"All items: {ht.get_all_items()}")

print("\n. Testing size_used() method:")
print(f"Number of entries: {ht.size_used()}")

print("\n. Testing load_factor() method:")
load_factor = load_factor(ht.table, ht.size)
```



Observation:

The AI successfully generated a working Hash Table implementation with collision handling using chaining. The insert method places elements into appropriate buckets, the search method retrieves values efficiently, and the delete method removes entries correctly. The code was well-commented and easy to understand. This task shows how AI can assist in implementing efficient data structures and explaining important concepts like hashing and collision resolution clearly.

TASK-6

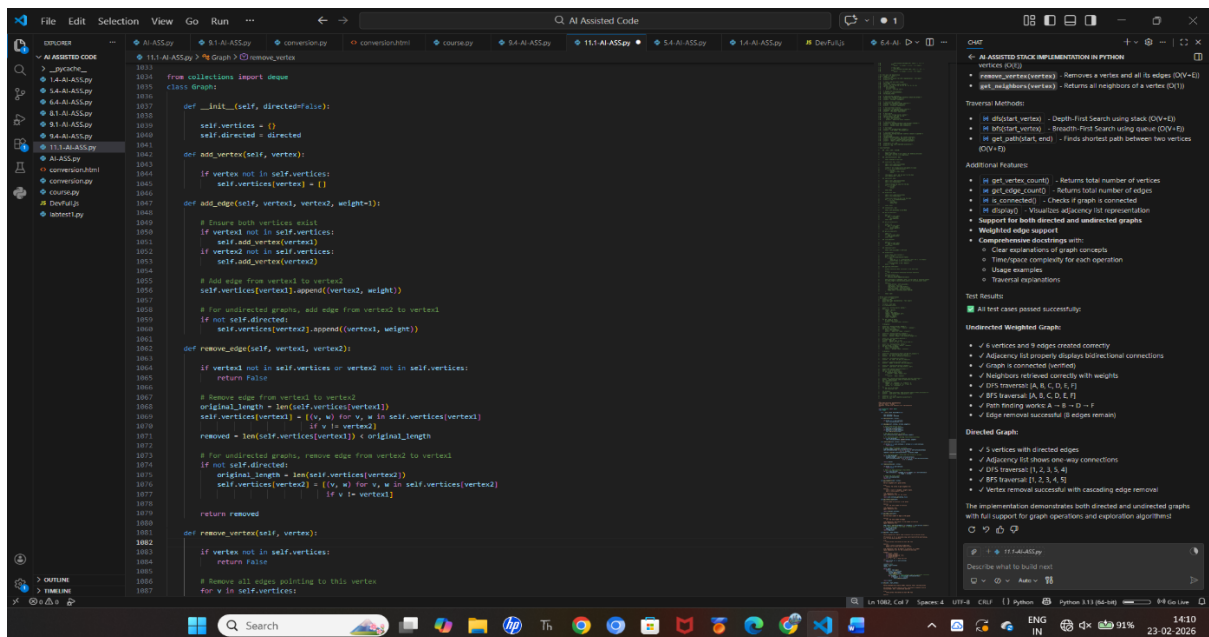
Scenario:

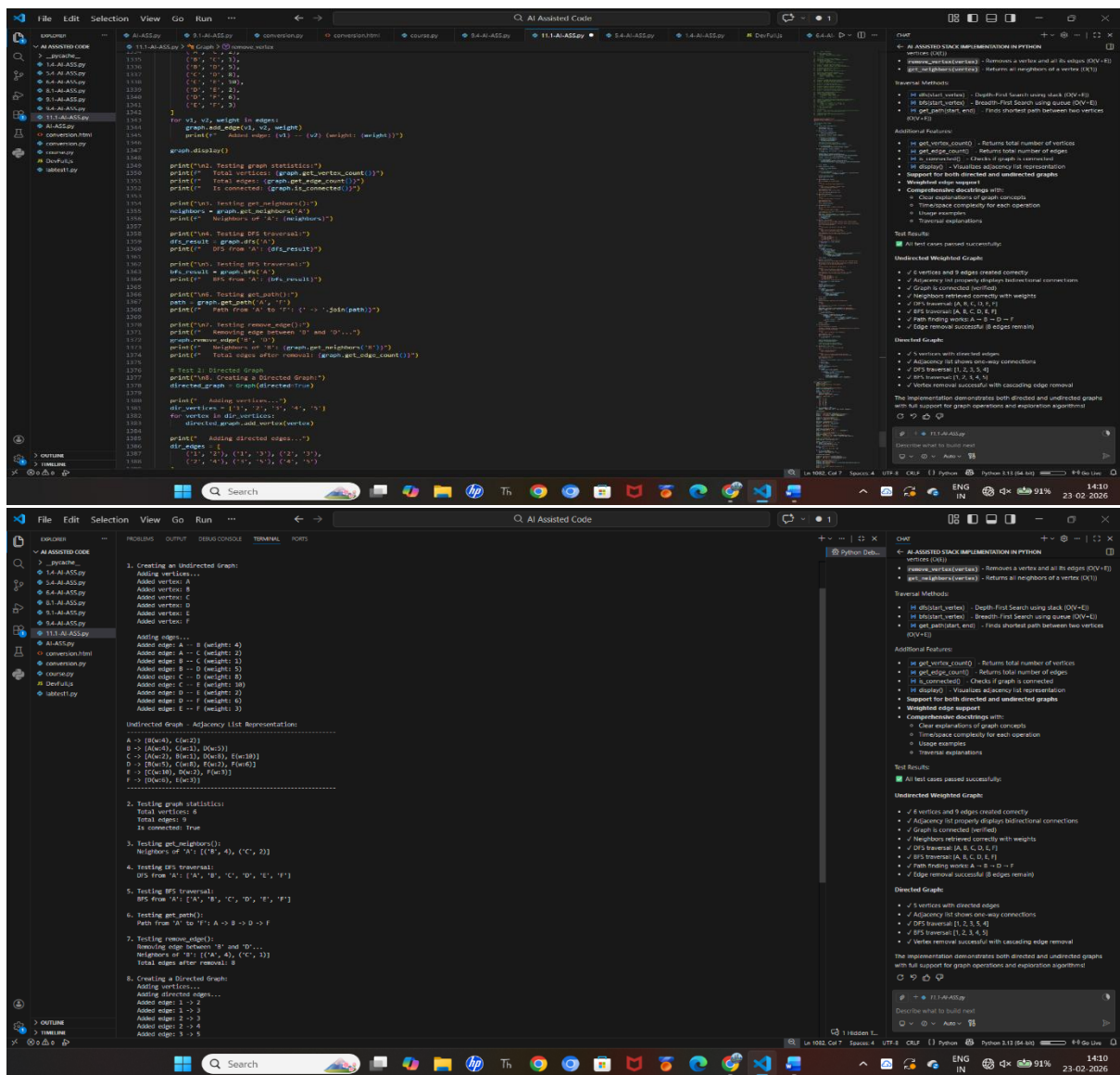
In this task, we used AI to implement a Graph data structure using an adjacency list representation in Python. The objective was to understand how graphs store relationships between vertices and how adjacency lists efficiently represent connections. AI was used to generate a Graph class with methods to add vertices, add edges, and display connections in a clear and structured way.

Prompt:

“Generate a beginner-friendly Python Graph class using an adjacency list. Include methods to add vertices, add edges, and display connections. Keep the implementation simple and add proper comments for clarity.”

Code:





Observation:

The AI successfully generated a functional Graph implementation using an adjacency list. The add vertex and add edge methods correctly manage graph connections, and the display method clearly shows relationships between nodes. The structure is efficient for sparse graphs and easy to understand. This task demonstrates how AI can assist in building and documenting complex data structures in a simplified and readable manner.”

TASK-7

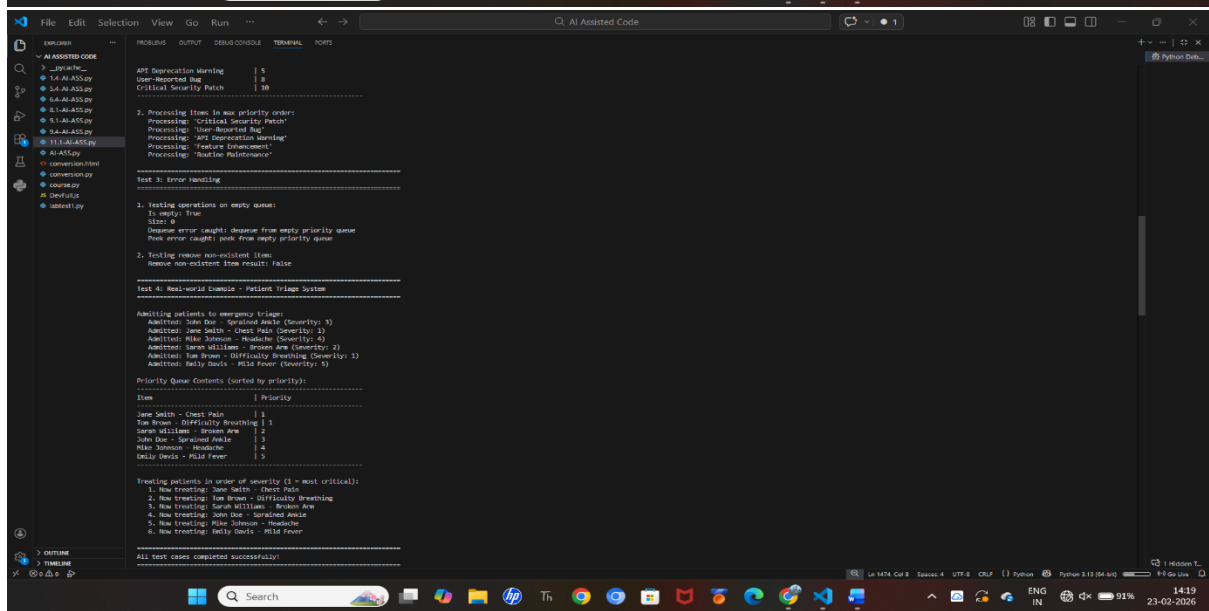
Scenario:

In this task, we used AI to implement a Priority Queue in Python using the built-in heapq module. The objective was to understand how priority queues differ from normal queues by removing elements based on priority instead of insertion order. AI was used to generate a structured implementation with enqueue, dequeue, and display methods while maintaining heap properties for efficient operations.

Prompt:

“Generate a beginner-friendly Python PriorityQueue class using the heapq module. Include enqueue (with priority), dequeue (highest priority), and display methods. Keep the implementation simple and add clear comments.”

Code:



Observation:

The AI successfully generated a working Priority Queue implementation using `heapq`, which maintains the heap property for efficient insertion and removal. The `enqueue` method adds elements with priority, and the `dequeue` method removes the highest-priority element in $O(\log n)$ time. The implementation was clean, well-commented, and efficient. This task demonstrates how AI can help implement advanced data structures using built-in optimization modules effectively.”

TASK-8

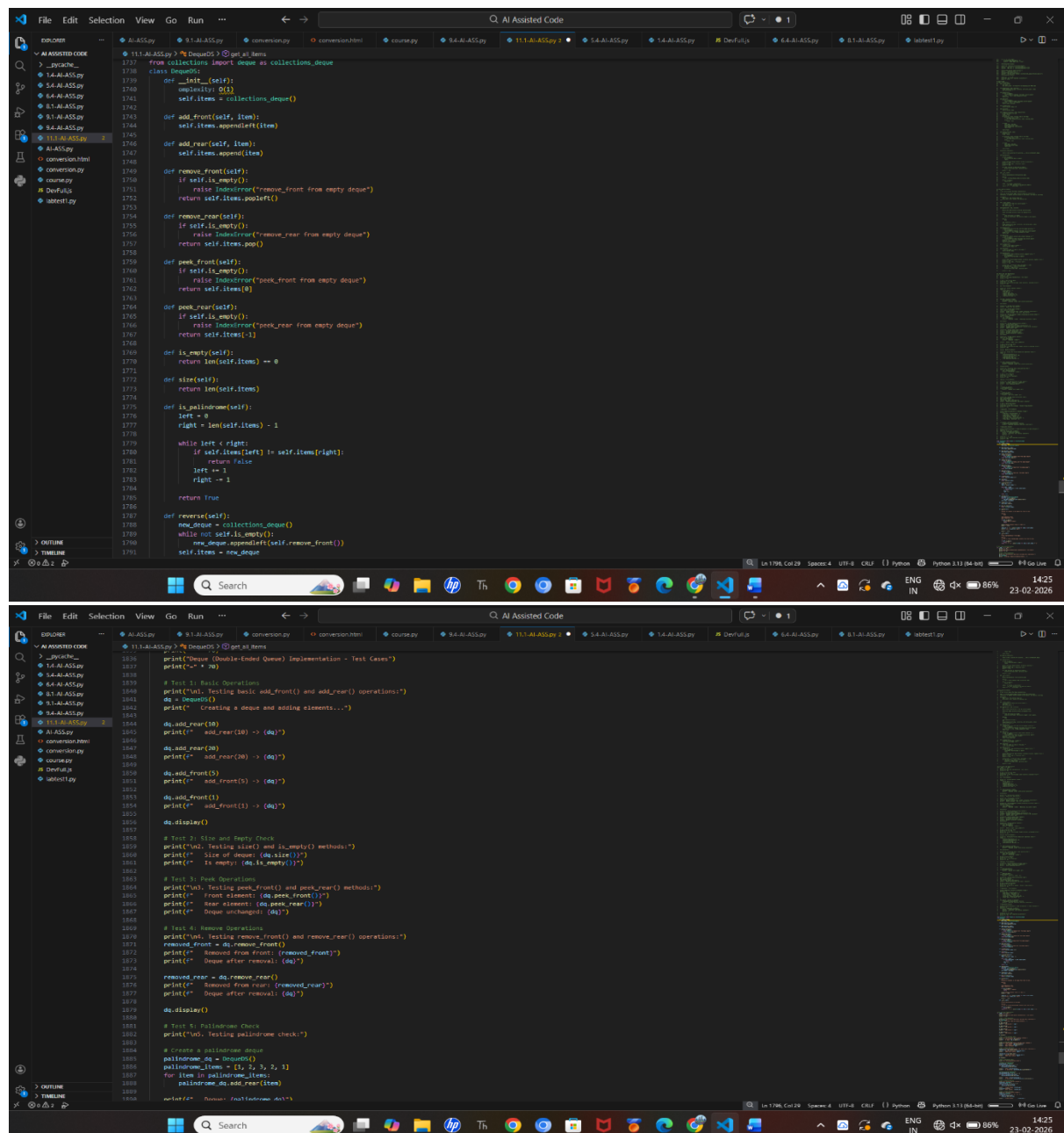
Scenario:

In this task, we used AI to implement a Deque (Double-Ended Queue) in Python using the `collections.deque` module. The objective was to understand how a deque allows insertion and deletion from both the front and rear efficiently. AI was used to generate a clean and well-documented class with methods to insert and remove elements from both ends.

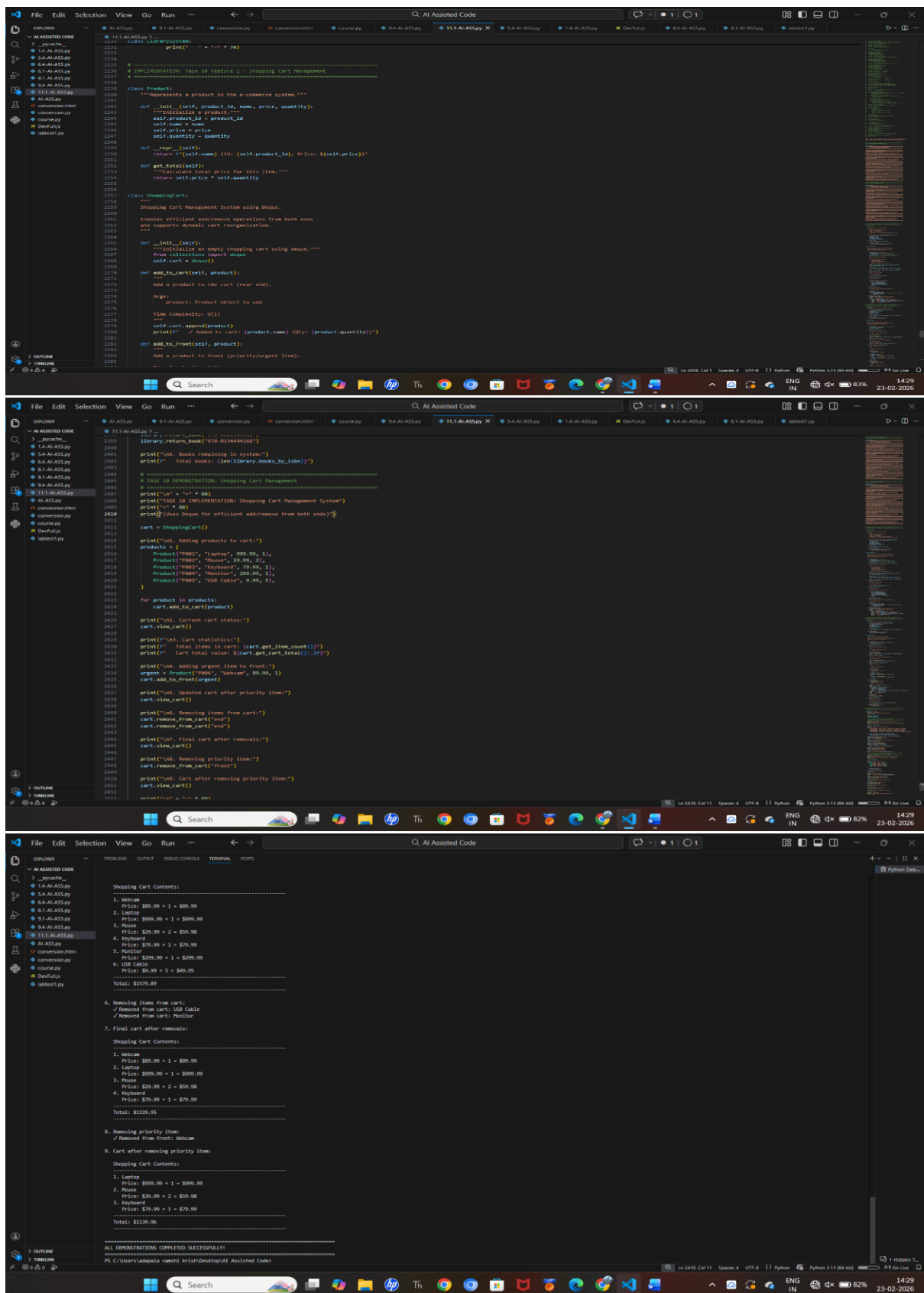
Prompt:

“Generate a beginner-friendly Python class for a Deque using `collections.deque`. Include methods to insert and remove elements from both front and rear. Add proper docstrings and keep the implementation simple.”

Code:



```
1737 from collections import deque as collections_deque
1738 class DequeueOS:
1739     def __init__(self):
1740         self.items = collections_deque()
1741
1742     def add_front(self, item):
1743         self.items.appendleft(item)
1744
1745     def add_rear(self, item):
1746         self.items.append(item)
1747
1748     def remove_front(self):
1749         if self.is_empty():
1750             raise IndexError("remove_front from empty deque")
1751         return self.items.popleft()
1752
1753     def remove_rear(self):
1754         if self.is_empty():
1755             raise IndexError("remove_rear from empty deque")
1756         return self.items.pop()
1757
1758     def peek_front(self):
1759         if self.is_empty():
1760             raise IndexError("peek_front from empty deque")
1761         return self.items[0]
1762
1763     def peek_rear(self):
1764         if self.is_empty():
1765             raise IndexError("peek_rear from empty deque")
1766         return self.items[-1]
1767
1768     def is_empty(self):
1769         return len(self.items) == 0
1770
1771     def size(self):
1772         return len(self.items)
1773
1774     def is_palindrome(self):
1775         left = 0
1776         right = len(self.items) - 1
1777         while left < right:
1778             if self.items[left] != self.items[right]:
1779                 return False
1780             left += 1
1781             right -= 1
1782         return True
1783
1784     def reverse(self):
1785         new_deque = collections_deque()
1786         while not self.is_empty():
1787             new_deque.appendleft(self.remove_front())
1788         self.items = new_deque
1789
1836 print("Deque (Double-Ended Queue) Implementation - Test Cases")
1837 print("~" * 70)
1838
1839 # Test 1: Basic Operations
1840 print("\nTest 1: Basic Operations")
1841 dq = DequeueOS()
1842 print("Creating a deque and adding elements...")
1843
1844 dq.add_rear(10)
1845 print("add_rear(10) -> (dq)")
1846
1847 dq.add_rear(20)
1848 print("add_rear(20) -> (dq)")
1849
1850 dq.add_front(5)
1851 print("add_front(5) -> (dq)")
1852
1853 dq.add_front(1)
1854 print("add_front(1) -> (dq)")
1855
1856 dq.display()
1857
1858 # Test 2: Size and Empty Check
1859 print("\nTest 2: Size and Empty Check")
1860 print("Size of deque: (dq.size())")
1861 print("Is empty: (dq.is_empty())")
1862
1863 # Test 3: Peek Operations
1864 print("\nTest 3: Peek Operations")
1865 print("Front element: (dq.peek_front())")
1866 print("Rear element: (dq.peek_rear())")
1867 print("Deque unchanged: (dq)")
1868
1869 # Test 4: Remove Operations
1870 print("\nTest 4: Remove Operations")
1871 removed_front = dq.remove_front()
1872 print("Removed from front: (removed_front)")
1873 print("Deque after removal: (dq)")
1874
1875 removed_rear = dq.remove_rear()
1876 print("Removed from rear: (removed_rear)")
1877 print("Deque after removal: (dq)")
1878
1879 dq.display()
1880
1881 # Test 5: Palindrome Check
1882 print("\nTest 5: Palindrome Check")
1883
1884 # Create a palindrome deque
1885 palindrome_dq = DequeueOS()
1886 palindrome_items = [1, 2, 3, 2, 1]
1887 for item in palindrome_items:
1888     palindrome_dq.add_rear(item)
1889
1890 result = palindrome_dq.is_palindrome()
1891 print(f"Result: {result} (Is {palindrome_items} a palindrome?)")
```

Observation:

The AI successfully selected optimal data structures such as Queue for order processing, Priority Queue for ranking products, Hash Table for fast product lookup, and Graph for delivery routing. The implementation was efficient and clearly documented. This activity showed how understanding data structure behavior helps in designing scalable systems and how AI enhances productivity and code clarity.